KIOUAZ NÉRIMEN TPA2
Fork : https://github.com/Nerimen81/R314-optimisation-site
Site déployé : https://nerimen81.github.io/R314-optimisation-site/
Captures : dans le dossier audit/. Je mets le nom de la capture entre parenthèses quand j'en parle.

Rapport

1) Comment j'ai mesuré

PageSpeed Insights (PSI), en Mobile et en Bureau. Pour l'« avant », j'ai analysé le site de référence du prof ; pour l'« après », mon site en ligne.

Onglet Réseau, en navigation privée avec le cache désactivé. Pour l'avant, j'ai mesuré dans Chrome sur localhost ; pour l'après, dans Safari sur le site en ligne.

Panneau Performance dans Chrome, avec le processeur ralenti ×4. J'ai rechargé la page puis scrollé pendant une vingtaine de secondes.

Lighthouse dans les DevTools (avant) : 54 en mobile et 69 en desktop (avant-lighthouse-mobile.png, avant-lighthouse-desktop.png).

2) Tableau comparatif

Captures : avant-reseau.png, apres-reseau.png, avant-psi-mobile.png, apres-psi-mobile.png, avant-psi-bureau.png, apres-psi-bureau.png. Les plus importantes sont en page 5.

Indicateur	Avant	Après
Poids total	~17,7 Mo (somme de la colonne Taille)	**852 ko**
Requêtes	22 (dont 1 en 404)	**14**, plus aucune erreur
FCP mobile / bureau	2,4 s / 0,9 s	2,4 s / **0,7 s**
LCP mobile / bureau	67,1 s / 11,9 s	**5,7 s / 1,3 s**
CLS mobile / bureau	0 / 0	0 / 0,021
TBT mobile / bureau	200 ms / 160 ms	350 ms / **120 ms**
Speed Index mobile / bureau	*20,1 s / 4,6 s*	***2,4 s / 0,7 s***
Score PSI mobile / bureau	**59 / 61**	**68 / 95**

Le score PSI au fil de mes corrections :

Étape	Mobile	Bureau
Site de référence	59	61
Après les corrections 1 à 13	56 (TBT 520 ms)	95

+ correction 14 (calendrier)	**68** (TBT 350 ms)	76 (**CLS 0,477**, intermediaire-psi-bureau-cls.png)
+ correction 15 (width/ height)	—	**95** (CLS 0,021)

Ce que j'en retiens :

- Le LCP, c'est le temps avant que le plus gros élément (ici la première photo) s'affiche. Le TBT, c'est le temps où la page est bloquée par le JavaScript et ne réagit pas. Le CLS, c'est à quel point la page « saute » pendant le chargement.

- En bureau, le score passe de 61 à 95 : le site est maintenant dans le vert.TBT

- Mobile : la première photo s'affiche 12 fois plus vite (67 s → 5,7 s), mais le TBT augmente de 200 à 350 ms malgré la suppression de JavaScript bloquant. Comme il varie fortement d'un test à l'autre, les 200 ms initiaux étaient peut-être un résultat exceptionnel : un seul test ne suffit pas pour conclure.

- CLS : il atteint 0,477 après l'allègement des images. Celles-ci, désormais plus rapides, arrivent pendant la mesure et décalent le texte, car leur taille n'était pas réservée à l'avance. La correction 15 (width et height) réserve cet espace et ramène le CLS à 0,021.

- Police : le faible CLS restant vient probablement du remplacement de la police par défaut par Poppins avec display=swap. Ce compromis est préférable à display=block, qui laissait le texte invisible pendant le chargement.

- LCP mobile : il reste à 5,7 s malgré une photo réduite à 42 Ko. Le problème vient probablement de son opacity: 0 initial : elle ne devient visible qu'après l'exécution de app.js. Sur mobile, le processeur simulé étant lent, la photo attend le JavaScript avant de s'afficher.

3) Les problèmes que j'ai trouvés

Partie 1 - Réseau, images, CSS

Fichier	Problème	Preuve	Correction	Gain mesuré
index.html	Le fichier polyfills.js n'existe pas (erreur 404), et il bloque l'affichage le temps que le navigateur le cherche	ligne rouge dans avant-reseau.png	j'ai supprimé la ligne	–1 requête, plus d'erreur
index.html	Une ligne prépare une connexion à un site (jsdelivr) que la page n'utilise jamais	jsdelivr absent de l'onglet Réseau	supprimée	—
index.html	img6.jpg?v=2 est préchargée, mais comme l'adresse est différente de img6.jpg, le navigateur télécharge l'image deux fois	2 lignes de 3 391 ko	supprimé	–3,3 Mo, –1 requête
img/img1…6.jpg	Les photos font jusqu'à 3968 px de large, alors qu'elles s'affichent en ~388 px	772 ko à 3,4 Mo chacune	redimensionnées en 1200 px avec Squoosh	img1 : 2 524 → 42 ko, img6 : 3 391 → 125 ko

hero-bg.jpg	L'image de fond du hero n'est visible qu'à 4 %, mais elle pèse 3,3 Mo	3 391 ko	gardée mais réduite en 480 px	3 391 → 15 ko
favicon.png	Le favicon fait 512 px alors qu'il s'affiche en tout petit	787 ko	réduit en 32 px	787 → ~1,5 ko
styles.css → extra.css → theme-legacy.css	Les fichiers CSS s'appellent l'un après l'autre (@import) : le navigateur doit attendre chaque fichier pour découvrir le suivant. C'est une **chaîne critique**	colonne Initiateur en escalier	j'ai gardé les 2 règles utiles dans styles.css et supprimé les 2 autres fichiers	–3 requêtes
styles.css	~1 200 règles CSS (.redundant-…) qui ne servent à rien	onglet Coverage + recherche dans le code	supprimées	92 → ~5 ko
police Poppins	Le texte est invisible tant que la police n'est pas chargée, une graisse ne sert pas, et la police est téléchargée deux fois	3 fichiers de police	display=swap, 2 graisses au lieu de 3	3 → 2 polices
index.html	Les images n'ont pas de taille dans le HTML, donc la page saute quand elles arrivent	CLS 0,477	ajout de width et height	CLS 0,477 → 0,021

Partie 2 - JavaScript (panneau Performance, onglet Bottom-Up)

Fichier	Problème	Preuve (Bottom-Up)	Correction
ui-kit.min.js	En dépliant le fichier, j'ai trouvé une boucle qui bloque la page 620 ms au chargement sans rien faire d'utile, et un « observateur » qui relit le style de toute la page à chaque changement	615 ms + 860 ms	app.js n'utilisait que 4 petites fonctions de cette librairie : je les ai réécrites moi-même et j'ai supprimé la librairie
app.js (revealCards)	À chaque scroll, le code mesure toutes les cartes juste après en avoir modifié une, ce qui oblige le navigateur à tout recalculer à chaque fois (**reflow forcé**)	3 149 ms	je fais d'abord toutes les mesures, puis toutes les modifications
app.js (buildKeywordIndex)	La liste des mots-clés est reconstruite entièrement à chaque mot ajouté (innerHTML +=)	338 ms	je construis la liste à part, puis je l'ajoute en une seule fois
app.js (calendrier)	Même problème que le 11, mais sur les 371 cases du calendrier	2 619 ms	d'abord toutes les mesures, puis toutes les modifications

Ce que j'ai compris, c'est que les problèmes 10 et 11 s'aggravaient entre eux : chaque style **modifié par revealCards déclenchait le MutationObserver de ui-kit, qui relisait toute la page**. (Petite erreur de ma part dans l'historique : les corrections 10 et 12 sont dans le même commit.)

Partie 3 - Le problème conditionnel

Fichier	Problème	Preuve / coût	Correction
js/ analytics.js	Le code est volontairement illisible. En décodant les textes cachés avec atob() dans la console, j'ai compris qu'il ne se lance que sur un écran de plus de 1024 px, après 6,5 secondes, puis toutes les 2,6 secondes. À chaque fois, il bloque la page un peu plus longtemps (de 46 ms jusqu'à 430 ms)	4 545 ms au total, blocs rouges réguliers dans la frise	la boucle ne servait qu'à attendre : maintenant le calcul se fait une seule fois

Pourquoi on ne le voit pas dans un audit classique : en profil mobile, l'écran fait moins de 1024 px, et un audit court s'arrête avant les 6,5 s. Ça explique l'écart entre mobile et desktop, et pourquoi le TBT desktop passe de 68 à 672 ms d'un essai à l'autre.

Ce que montre le panneau Performance après (apres-performance.png, apres-performance-bottomup.png)

Activité	Avant	Après
ui-kit (boucle + observateur)	~1,5 s	disparu
analytics.js	4 545 ms	disparu
revealCards	3 149 ms	1 333 ms
getBoundingClientRect	2 619 ms	245 ms
Recalculate style	2 123 ms	2 428 ms

Dans la frise, il n'y a **plus les blocs rouges qui revenaient régulièrement** : maintenant, le processeur ne travaille que pendant que je scrolle. Les deux gros coupables, ui-kit et analytics, ont disparu du classement Bottom-Up.

Par contre, **Recalculate style n'a pas baissé**, et je pense savoir pourquoi : revealCards tourne encore à chaque mouvement du scroll. Elle relit la hauteur des cartes et réécrit l'opacité des 371 cases du calendrier à chaque fois. J'ai corrigé l'ordre des lectures et des écritures (plus de reflow forcé), mais il reste trop de travail à chaque scroll. **C'est donc une correction partielle.**

4. Iso-rendu

J'ai comparé deux captures de la page entière (isorendu-avant.png, isorendu-apres.png) : les textes, les couleurs, les ombres, la texture du hero, les photos, le filtre et le calendrier sont identiques.

La seule différence que j'ai remarquée, c'est un peu plus d'espace sous les photos de la galerie sur le site d'origine. D'après ce que j'ai compris, la hauteur des cartes est calculée en JavaScript, et sur l'original ce calcul se faisait pendant que les images chargeaient, sans taille connue. Le

résultat changeait donc selon le moment du calcul. Avec width et height, la taille est connue tout de suite.

Sur les pages de détail (img1.html à img6.html), j'ai ajouté width:100% pour que l'image ne dépasse plus de l'écran. Avant, on devait faire défiler la page sur le côté pour voir toute l'image, ce qui est un bug d'affichage (point A1 du guide).

6. Conclusion

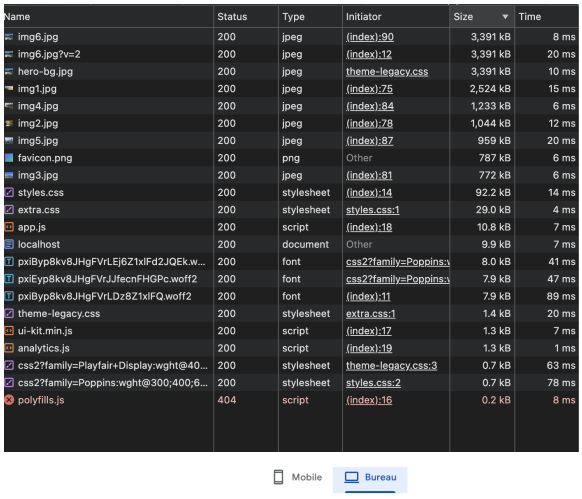
Le site est passé de ~17,7 Mo à 852 ko et de 22 à 14 requêtes. Le score PSI est passé de 61 à 95 en bureau et de 59 à 68 en mobile. Les images m'ont fait gagner le plus de poids, mais ce qui faisait vraiment ramer le site, c'était le JavaScript : une boucle bloquante dans une librairie, des reflows forcés au scroll, et un script d'analytics qui ne se déclenche que sur grand écran après 6,5 s. Je n'aurais pas trouvé ce dernier sans un enregistrement long et sans comparer mobile et desktop.

Limites de mon travail

- PageSpeed n'affiche pas de données d'utilisateurs réels, parce que le site n'a pas de visiteurs. Mes résultats viennent donc seulement de tests simulés.
- J'ai fait ma mesure Réseau « avant » en local, et un serveur local ne compresse pas les fichiers. Le CSS y paraissait donc plus lourd qu'en vrai.
- Les scores changeaient beaucoup d'un test à l'autre, surtout le TBT. Je n'ai pas toujours eu le temps de relancer 3 fois, donc je me suis surtout fiée aux grandes tendances.

Captures principales : (toutes les captures sont dans le dossier audit)

Avant



Après

